

SimuPi

Manual de usuario

Simulador de Raspberry Pi para IoT

Cloud Computing & Cloud Robotics

Facultad de Informática - Universidad Nacional de La Plata

Desarrollado por: Sebastián Rodríguez Eguren y Claude IA

Guía para estudiantes: puesta en marcha, uso de la interfaz web y solución de problemas.

Contenido

1. ¿Qué es SimuPi?

2. Puesta en marcha

3. Recorrido por la pantalla

4. Escenarios: elegir qué simular

5. GPIO Inputs: pines de entrada

6. GPIO Outputs: pines de salida

7. Sensores

8. Pestañas inferiores

9. Configuración: crear tus propios escenarios

10. Qué se guarda al cerrar SimuPi

11. Conectar otro programa por MQTT

12. Problemas frecuentes

A. Apéndice: referencia rápida de la API

1. ¿Qué es SimuPi?

SimuPi es un **simulador de Raspberry Pi** para trabajos de IoT. Genera lecturas de sensores, simula pines GPIO y publica todo por MQTT, igual que lo haría una Raspberry Pi real con su kit de sensores. Todo corre en un único programa: no hace falta instalar Mosquitto, Docker ni ningún otro servicio.

La idea es que puedas **desarrollar y probar tus flujos de Node-RED sin hardware**. Los flujos que armes contra SimuPi funcionan sin cambios cuando pasen a la Raspberry Pi física.

Al ejecutarlo se levantan dos servicios en tu computadora:

Servicio	Dirección	Para qué sirve
Interfaz web	<code>http://localhost:5000</code>	Ver y manipular sensores y pines desde el navegador.
Broker MQTT	<code>localhost:1883</code>	Recibir las lecturas y mandar órdenes desde Node-RED u otro cliente MQTT.

Este manual explica **cómo usar la interfaz web**, pantalla por pantalla. La integración con Node-RED y los ejercicios están en el tutorial de laboratorio aparte.

2. Puesta en marcha

Antes de empezar

- Los puertos **5000** (web) y **1883** (MQTT) tienen que estar libres en tu computadora.
- Un navegador moderno (Chrome, Firefox o Edge).
- Conexión a Internet: la interfaz descarga la librería del gráfico (Chart.js) al abrirse. Sin Internet todo funciona, salvo el gráfico del Histórico.

Ejecutar SimuPi

Si tu docente te entregó el ejecutable, alcanza con abrirlo. Si tenés que compilarlo, necesitás **Go 1.24 o superior**.

Sistema	Cómo ejecutarlo
Linux / macOS	<code>./simupi</code>
Windows	<code>simupi.exe</code> (desde PowerShell o CMD, dentro de la carpeta)
Compilar y ejecutar (con Go)	<code>make run</code> o <code>go build -o simupi . && ./simupi</code>

Si el puerto 5000 está ocupado, podés elegir otro con la variable `PORT`:

```
PORT=8080 ./simupi           # Linux / macOS
$env:PORT=8080; .\simupi.exe  # Windows (PowerShell)
```

La terminal muestra una línea como esta cuando todo arrancó bien:

```
SimuPi dev – http://localhost:5000 | MQTT on :1883 | escenario: garage
```

Abrir la interfaz

- 1 Dejá la terminal abierta: mientras esté corriendo, SimuPi está activo.
- 2 Abrió el navegador y entrá a <http://localhost:5000>.
- 3 Para verificar que el servidor responde podés abrir <http://localhost:5000/api/health>: tiene que mostrar `{"status": "ok"}`.
- 4 Para detenerlo, volvé a la terminal y presioná **Ctrl + C**.

Orden de inicio

Iniciá siempre SimuPi **antes** que Node-RED. El broker MQTT de SimuPi tiene que estar disponible cuando Node-RED intente conectarse.

3. Recorrido por la pantalla

Al abrir la página vas a ver el panel principal, dividido en cinco zonas. Los datos se actualizan solos cada 2 segundos; no hace falta recargar la página.

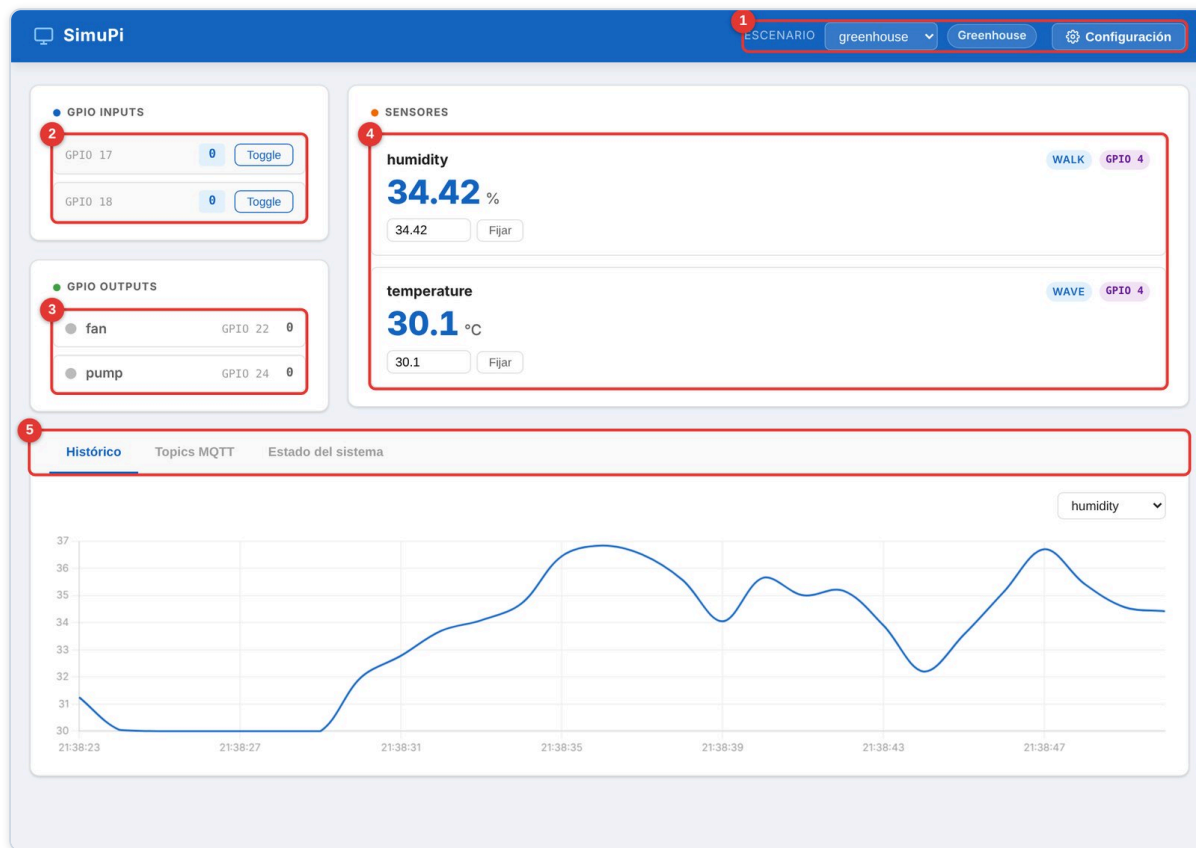


Figura 1. Panel principal de SimuPi con el escenario *greenhouse* cargado.

- 1 **Barra superior:** elegir el escenario y abrir la configuración.
- 2 **GPIO Inputs:** pines de entrada (botones, sensores digitales). Los controlás vos desde acá.
- 3 **GPIO Outputs:** pines de salida (relés, LEDs, motores). Los controla tu programa por MQTT.
- 4 **Sensores:** valor actual de cada sensor simulado y sus controles.
- 5 **Pestañas:** Histórico (gráfico), Topics MQTT y Estado del sistema.

4. Escenarios: elegir qué simular

Un **escenario** define qué sensores y qué pines existen. SimuPi trae tres escenarios listos y vos podés crear los tuyos (ver [Configuración](#)).



Figura 2. Barra superior: selector de escenario, escenario activo y botón de Configuración.

- 1 **Selector de escenario:** lista de escenarios disponibles.
- 2 **Escenario activo:** nombre del escenario que está corriendo en este momento.
- 3 **Configuración:** abre el editor de escenarios.

Escenario	Sensores	GPIO inputs	GPIO outputs
garage	distance (10–400 cm)	17, 18, 23	22 → door, 24 → light
greenhouse	temperature (15–40 °C), humidity (30–90 %)	17, 18	22 → fan, 24 → pump
smart_home	temperature (18–30 °C), power (100–2500 W)	17, 18, 23	22 → living_light, 24 → heater, 25 → alarm

Cambiar de escenario

- 1 Abrió el selector **Escenario** y elegí uno de la lista.
- 2 El cambio es inmediato: no hace falta reiniciar SimuPi.
- 3 Verificá en el cuadro del escenario activo (2) que muestre el nombre esperado.

Qué pasa al cambiar

Se reemplazan los sensores y los pines, y el historial del gráfico **empieza de cero**. Si estabas probando un flujo en Node-RED, los tópicos pueden cambiar (por ejemplo `sensor/distance` deja de existir si pasás de garage a greenhouse).

El selector no siempre refleja el escenario activo

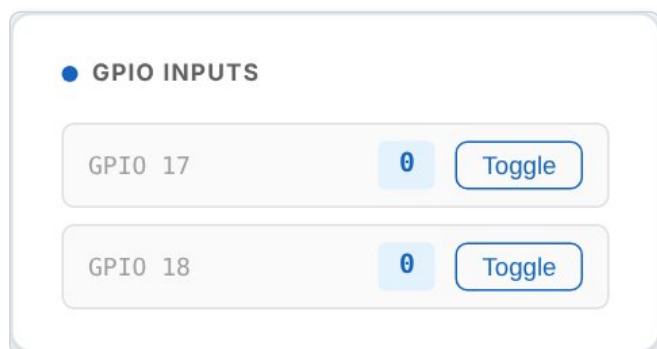
Si recargás la página, el selector vuelve a mostrar el primer escenario de la lista aunque el activo sea otro. **El escenario real es el que dice el cuadro que está al lado (2).** Si querés estar seguro, volvé a elegirlo en el selector.

5. GPIO Inputs: pines de entrada

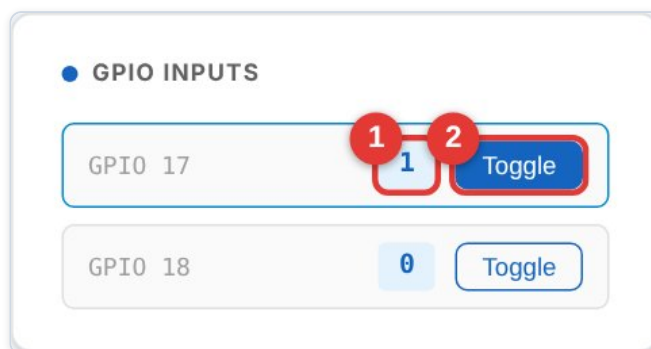
Un **GPIO Input** simula algo que *entra* a la Raspberry Pi: un pulsador, un sensor de puerta, un detector de presencia. Su valor es siempre **0** o **1**. En este panel el que manda sos vos: SimuPi publica el valor por MQTT en el tópico `gpio/in/<pin>`.

Cambiar el valor de un pin

- 1 Buscá el pin en el panel **GPIO Inputs** (por ejemplo **GPIO 17**).
- 2 Mirá el valor actual en el recuadro celeste (0 o 1).
- 3 Hacé clic en **Toggle**: el valor pasa de 0 a 1, o de 1 a 0.



Antes: GPIO 17 vale 0.



Después de hacer clic en Toggle: GPIO 17 vale 1 (1 = valor, 2 = botón).

Alias

Si el escenario define un nombre para el pin, aparece al lado del número (por ejemplo *puerta*). Los alias se definen en la configuración del escenario.

6. GPIO Outputs: pines de salida

Un **GPIO Output** simula un actuador: un relé, un LED, un ventilador, una bomba. Acá **no hay botones**: el pin cambia cuando **tu programa le manda una orden por MQTT**. Es lo mismo que pasaría con la Raspberry Pi real.

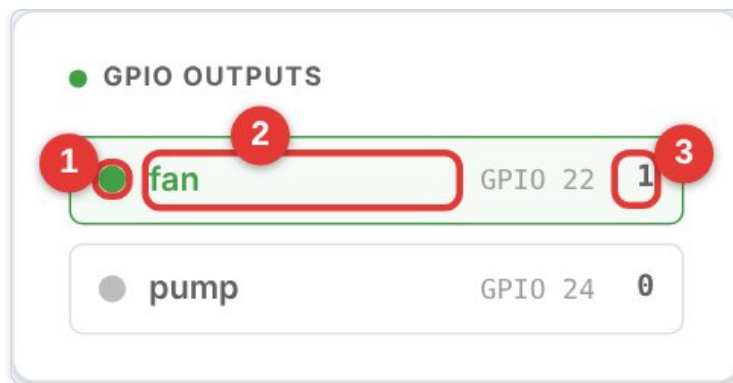


Figura 3. El pin 22 (*fan*) recibió un 1: la fila se pone verde y el LED se enciende.

- 1 **LED:** gris = apagado (0), verde = encendido (1).
- 2 **Nombre:** el alias del pin, o *GPIO n* si no tiene alias.
- 3 **Valor actual:** 0 o 1.

Cómo encender o apagar una salida

Publicá un mensaje MQTT en el tópico `gpio/out/<pin>` con payload `1` (encender) o `0` (apagar).

Desde	Cómo
Node-RED	Nodo mqtt out : servidor <code>localhost:1883</code> , tópico <code>gpio/out/22</code> , payload <code>1</code> .
Terminal (si tenés Mosquitto instalado)	<code>mosquitto_pub -h localhost -t gpio/out/22 -m 1</code>

Cada tarjeta de la pestaña **Topics MQTT** te muestra el tópico exacto de cada pin.

Si la salida no cambia

Revisá que el pin exista en el escenario activo (por ejemplo, el 25 solo existe en *smart_home*), que el payload sea exactamente `0` o `1` y que el tópico sea `gpio/out/` + número de pin. SimuPi ignora sin avisar los mensajes que no cumplen eso.

7. Sensores

Cada sensor simulado aparece en una tarjeta con su valor actual. SimuPi publica ese valor por MQTT en `sensor/<nombre>` con dos decimales, una vez por intervalo (normalmente cada 1 segundo).



Figura 4. Tarjeta de un sensor.

- 1 **Nombre** del sensor; es el que se usa en el t pico MQTT (`sensor/humidity`).
- 2 **Modo** de simulaci n: c mo se genera la se al (ver tabla m s abajo).
- 3 **Pin GPIO**: a qu  pin de la Raspberry Pi real se conectar a. Es solo informativo.
- 4 **Valor actual** y su unidad.
- 5 **Control manual**: campo num rico y bot n **Fijar** (ver m s abajo).

Modos de simulaci n

El modo define la forma de la se al. Cada sensor usa el suyo.

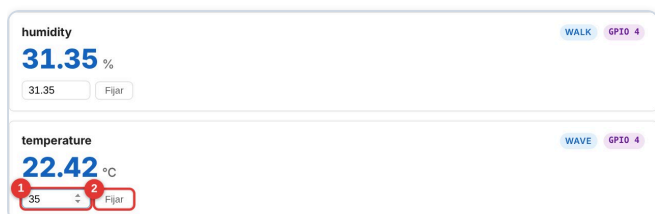
Modo	Comportamiento	Parecido a...
wave	Onda senoidal suave entre el m�nimo y el m�ximo.	Temperatura ambiente d�a/noche
triangle	Sube y baja en l�nea recta, de forma sim�trica.	Tanque que se llena y se vac�a
sawtooth	Sube en l�nea recta y vuelve de golpe al m�nimo.	Llenado continuo con reinicio
square	Alterna entre m�nimo y m�ximo.	Se�al digital, presencia
walk	Camina al azar: cada lectura cambia poco respecto de la anterior.	Humedad
ramp	Sube y baja en escalones fijos.	Distancia de un objeto que se mueve
random	Valor al azar entre m�nimo y m�ximo, sin continuidad.	Consumo el�ctrico instant�neo
slider	No cambia solo: lo mov�s vos con una barra deslizante.	Un potenc�metro

Fijar un valor a mano

Sirve para probar c mo reacciona tu flujo ante un valor concreto (por ejemplo, una temperatura alta que active un ventilador).

- 1 En la tarjeta del sensor, escrib  el valor en el campo num rico.

- 2 Hacé clic en **Fijar**.
- 3 La tarjeta se pone naranja, aparece la etiqueta **FIJADO** y el sensor queda en ese valor, sin cambiar.
- 4 Para volver a la simulación normal, hacé clic en **Liberar**.



Antes de fijar: se escribió 35 en el campo.



Después: el sensor quedó fijado en 35 (1 = etiqueta, 2 = valor, 3 = botón Liberar).

El valor fijado se mantiene

El valor fijado se publica por MQTT en cada intervalo y se guarda con el estado, así que **sigue fijado aunque reinicies SimuPi** hasta que lo liberes.

Sensores de tipo slider

En un sensor **slider** no hay campo ni botón: aparece una barra con el mínimo y el máximo en los extremos. Arrastrala; el valor se muestra en vivo mientras la movés y se envía al soltar.

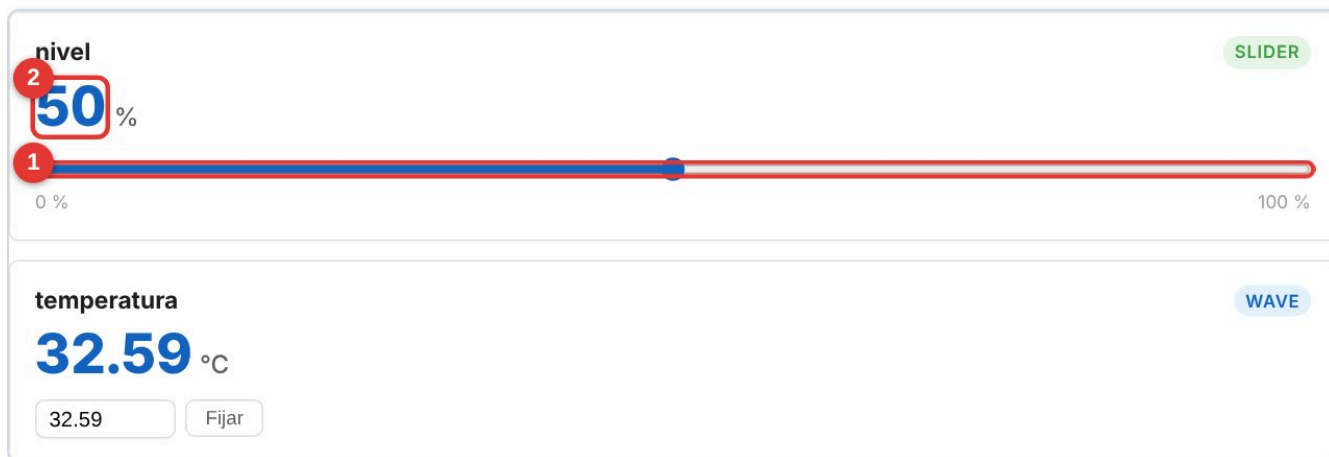


Figura 5. Sensor en modo slider: (1) barra deslizando, (2) valor actual.

Simular fallas de un sensor

Los sensores reales fallan. Para probar que tu flujo lo detecta, SimuPi puede simular dos fallas:

Falla	Qué hace	Cómo se ve
stuck	El valor queda congelado ; sigue publicando siempre el mismo número.	Borde y etiqueta naranjas
offline	El sensor deja de publicar : no llegan más mensajes por MQTT.	Tarjeta atenuada y etiqueta roja

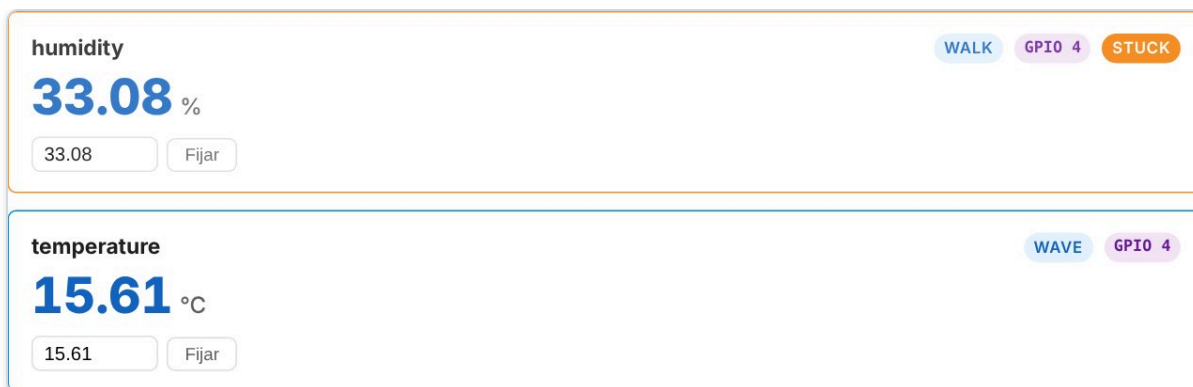


Figura 6. Sensor *humidity* con la falla *stuck* activa.

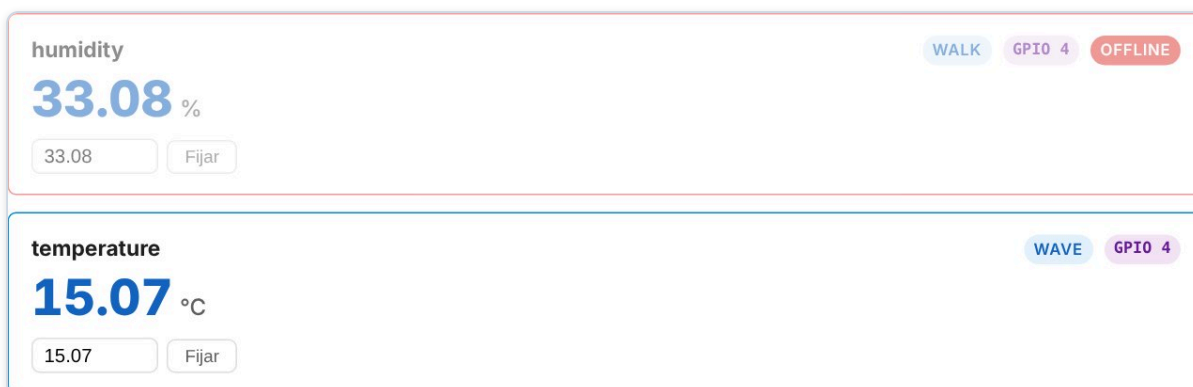


Figura 7. Sensor *humidity* con la falla *offline* activa.

Las fallas **no se activan desde la interfaz**: se piden al servidor con una llamada HTTP desde otra terminal. Reemplazá `humidity` por el nombre de tu sensor.

```
# Linux / macOS
curl -X POST http://localhost:5000/api/failure/humidity -H "Content-Type: application/json"
-d '{"mode":"stuck"}'

# Windows (PowerShell)
Invoke-RestMethod -Method Post -Uri http://localhost:5000/api/failure/humidity -ContentType
"application/json" -Body '{"mode":"stuck"}'
```

Los valores posibles de `mode` son `stuck`, `offline` y `clear`. Con `clear` el sensor vuelve a la normalidad.

8. Pestañas inferiores

Histórico

Gráfico en tiempo real de un sensor. Guarda las últimas **500 lecturas** de cada uno.

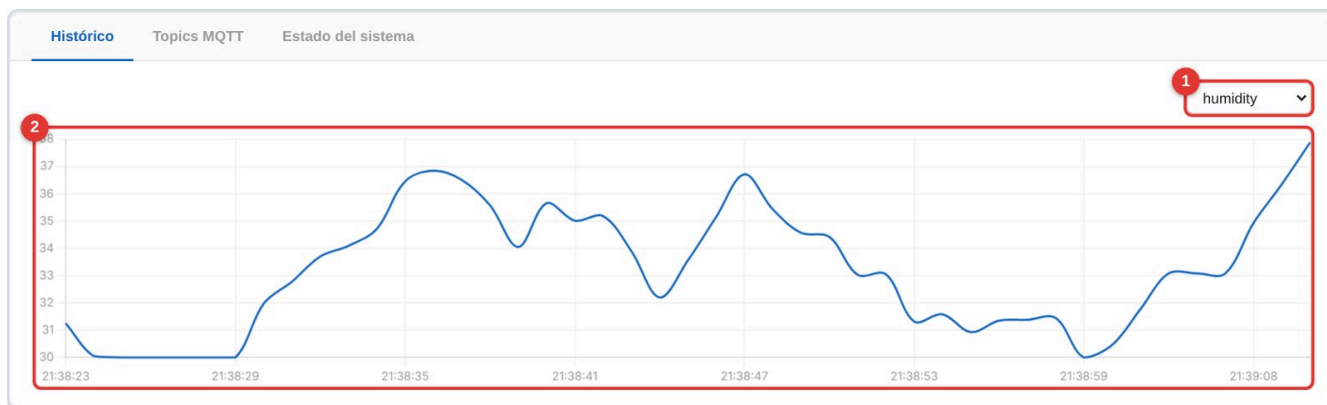


Figura 8. Pestaña Histórico: (1) selector de sensor, (2) gráfico.

- 1 Elegí el sensor en el selector (1).
- 2 Pasá el mouse sobre la línea para ver el valor y la hora exactos de cada punto.

Para **descargar el historial en CSV**, abrí en el navegador

<http://localhost:5000/api/export/<sensor>> (por ejemplo [/api/export/humidity](http://localhost:5000/api/export/humidity)).

El gráfico necesita Internet

La librería del gráfico se descarga de Internet al abrir la página. Si no tenés conexión, el panel queda vacío, pero los valores y el resto de la interfaz siguen funcionando.

Topics MQTT

Lista de todos los tópicos del escenario activo, con su valor actual. Es la forma más rápida de saber **qué tópico usar en Node-RED**.

The screenshot shows the 'Topics MQTT' tab of a web interface. At the top, there are three tabs: 'Historico', 'Topics MQTT', and 'Estado del sistema'. The 'Topics MQTT' tab is active. Below the tabs, there is a section titled 'PUBLICADOS POR EL SIMULADOR' with a red circle '1' next to it. This section contains a list of topics: 'sensor/humidity' (sensor · GPIO 4 · 39.5 %), 'sensor/temperature' (sensor · GPIO 4 · 16.68 °C), 'gpio/in/17' (GPIO input · valor 0), and 'gpio/in/18' (GPIO input · valor 0). Each topic has a 'Copiar' button next to it, with a red circle '2' highlighting the 'Copiar' button for 'sensor/humidity'. Below this section, there is another section titled 'RECIBIDOS POR EL SIMULADOR (NODE-RED → OUTPUTS)' with a green border. This section contains two topics: 'gpio/out/22' (GPIO output · fan · publicar 0 o 1) and 'gpio/out/24' (GPIO output · pump · publicar 0 o 1). Each topic has a 'Copiar' button next to it.

Figura 9. Pestaña Topics MQTT: (1) tópicos publicados por SimuPi (azul), (2) botón Copiar. Los de abajo (verde) son los que SimuPi recibe.

Tipo	Tópico	Contenido
Publicado por SimuPi	<code>sensor/<nombre></code>	Valor del sensor, por ejemplo <code>24.53</code>
Publicado por SimuPi	<code>gpio/in/<pin></code>	<code>0</code> o <code>1</code>
Recibido por SimuPi	<code>gpio/out/<pin></code>	<code>0</code> o <code>1</code> (vos lo enviás)

El botón **Copiar** pega el nombre del tópico en el portapapeles, listo para usarlo. En Node-RED podés suscribirte a todos los sensores a la vez con el tópico `sensor/#`.

Estado del sistema

Muestra el estado interno completo en formato JSON: valores de sensores, pines, modos, fallas y alias. Sirve para depurar cuando algo no se comporta como esperabas.

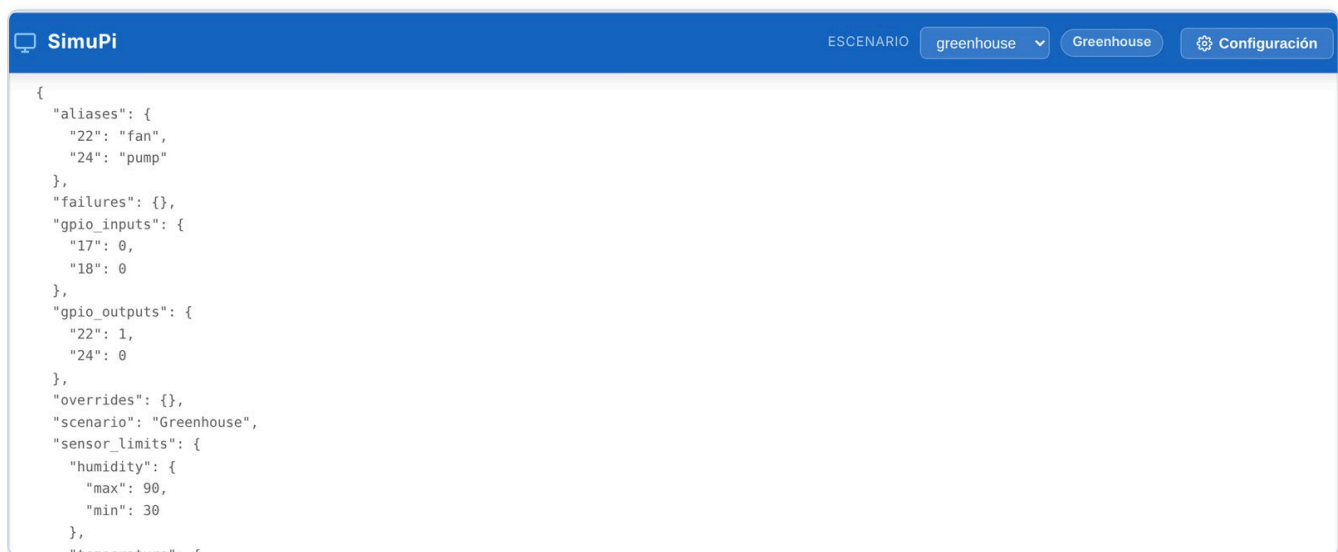


Figura 10. Pestaña Estado del sistema (se muestra solo la parte superior).

9. Configuración: crear tus propios escenarios

El botón **Configuración** abre un editor donde podés cambiar los sensores, los pines y los alias del escenario activo, o armar uno nuevo.

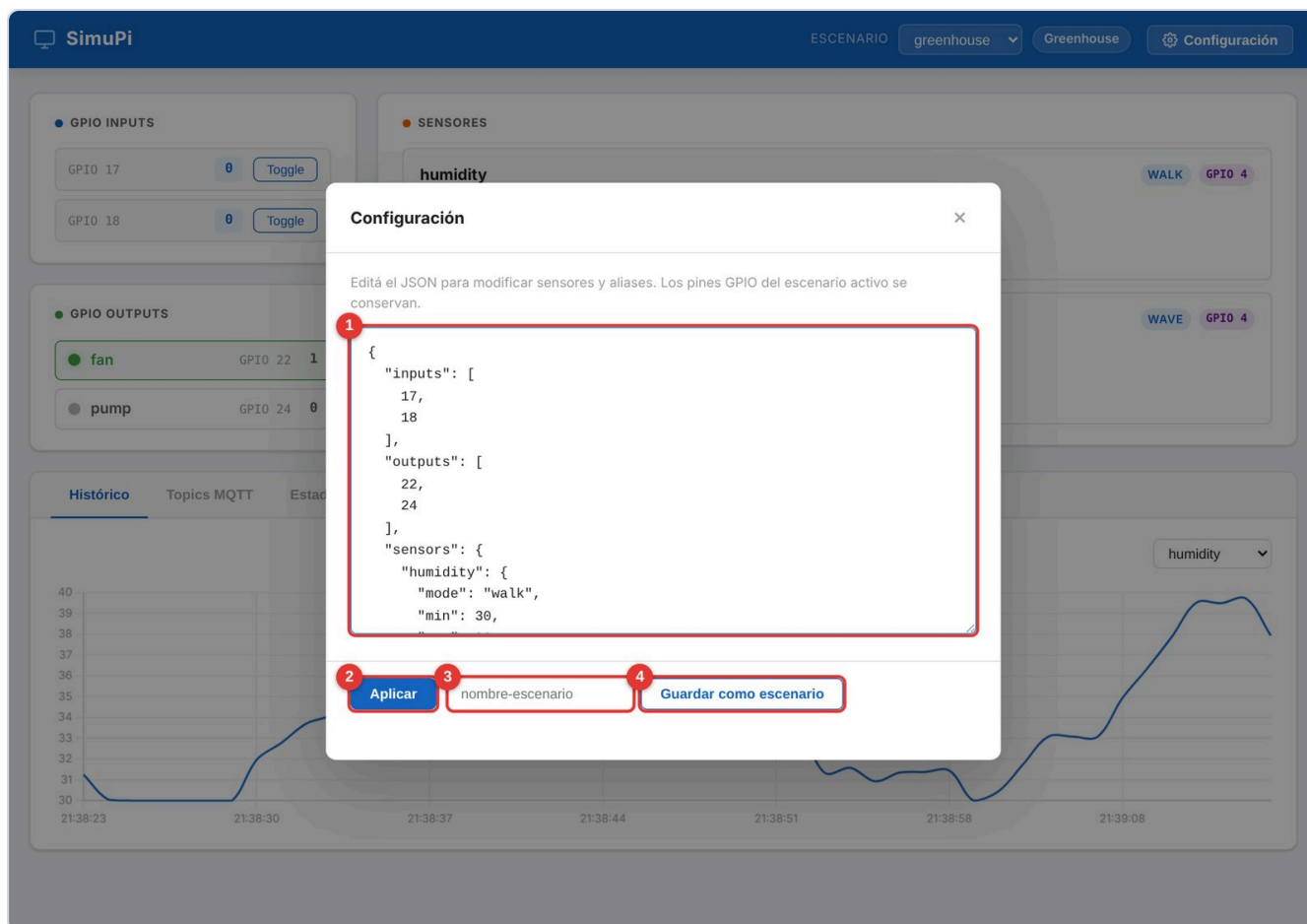


Figura 11. Ventana de Configuración: (1) editor JSON, (2) Aplicar, (3) nombre del nuevo escenario, (4) Guardar como escenario.

Estructura del JSON

```
{
  "inputs": [17, 18],
  "outputs": [22, 24],
  "alias": { "22": "ventilador", "24": "bomba" },
  "sensors": {
    "temperatura": { "mode": "wave", "min": 15, "max": 40, "interval": 1, "period": 60,
    "unit": "°C" },
    "nivel": { "mode": "slider", "min": 0, "max": 100, "interval": 1, "unit": "%" }
  }
}
```

Campo	Significado
inputs	Lista de números de pin que funcionan como entradas.
outputs	Lista de números de pin que funcionan como salidas.
alias	Nombre amigable para un pin: "número de pin": "nombre" .
sensors	Un sensor por entrada. La clave es el nombre (sale en el tópico <code>sensor/<nombre></code>).

Parámetros de cada sensor

Parámetro	Obligatorio	Descripción
<code>mode</code>	Sí	<code>wave</code> , <code>triangle</code> , <code>sawtooth</code> , <code>square</code> , <code>walk</code> , <code>ramp</code> , <code>random</code> o <code>slider</code> .
<code>min</code> , <code>max</code>	Sí	Rango del sensor. min tiene que ser menor que max.
<code>interval</code>	Recomendado	Cada cuántos segundos se publica una lectura (si falta, se usa 1).
<code>unit</code>	No	Unidad que se muestra (°C, %, cm...).
<code>period</code>	No	Duración de un ciclo en segundos, para <code>wave</code> , <code>triangle</code> , <code>sawtooth</code> y <code>square</code> (por defecto 60).
<code>duty</code>	No	Solo <code>square</code> : fracción del ciclo en valor alto, de 0 a 1 (por defecto 0,5).
<code>step</code>	No	Solo <code>walk</code> : cuánto puede cambiar el valor por lectura.
<code>pin</code>	No	Pin de la Raspberry Pi real donde iría el sensor. Solo informativo.

Aplicar o guardar

Botón	Qué hace
Aplicar	Usa la configuración ya, de forma temporal . No se guarda como escenario nuevo.
Guardar como escenario	La guarda con el nombre que escribas, la aplica y la agrega a la lista del selector. Se conserva aunque reinicies SimuPi.

Paso a paso: crear un escenario propio

- 1 Hacé clic en **Configuración**.
- 2 Editá el JSON (por ejemplo, agregá un sensor o cambiá los alias).
- 3 Para probarlo sin guardar, hacé clic en **Aplicar**. Aparece **Configuración aplicada** y la ventana se cierra sola.
- 4 Para conservarlo, escribí un nombre en el campo (3) y hacé clic en **Guardar como escenario**.
- 5 El nombre admite solo letras, números, guiones (-) y guiones bajos (_), hasta 50 caracteres. Sin espacios.

Configuración

×

Edita el JSON para modificar sensores y alias. Los pines GPIO del escenario activo se conservan.

```
"max": 40,
"interval": 1,
"period": 60,
"unit": "°C"
},
"nivel": {
  "mode": "slider",
  "min": 0,
  "max": 100,
  "interval": 1,
  "unit": "%"
}
}
```

Aplicar

nombre-escenario

Guardar como escenario

Escenario "mi-laboratorio" guardado

Figura 12. Escenario guardado: aparece el mensaje en verde debajo de los botones.

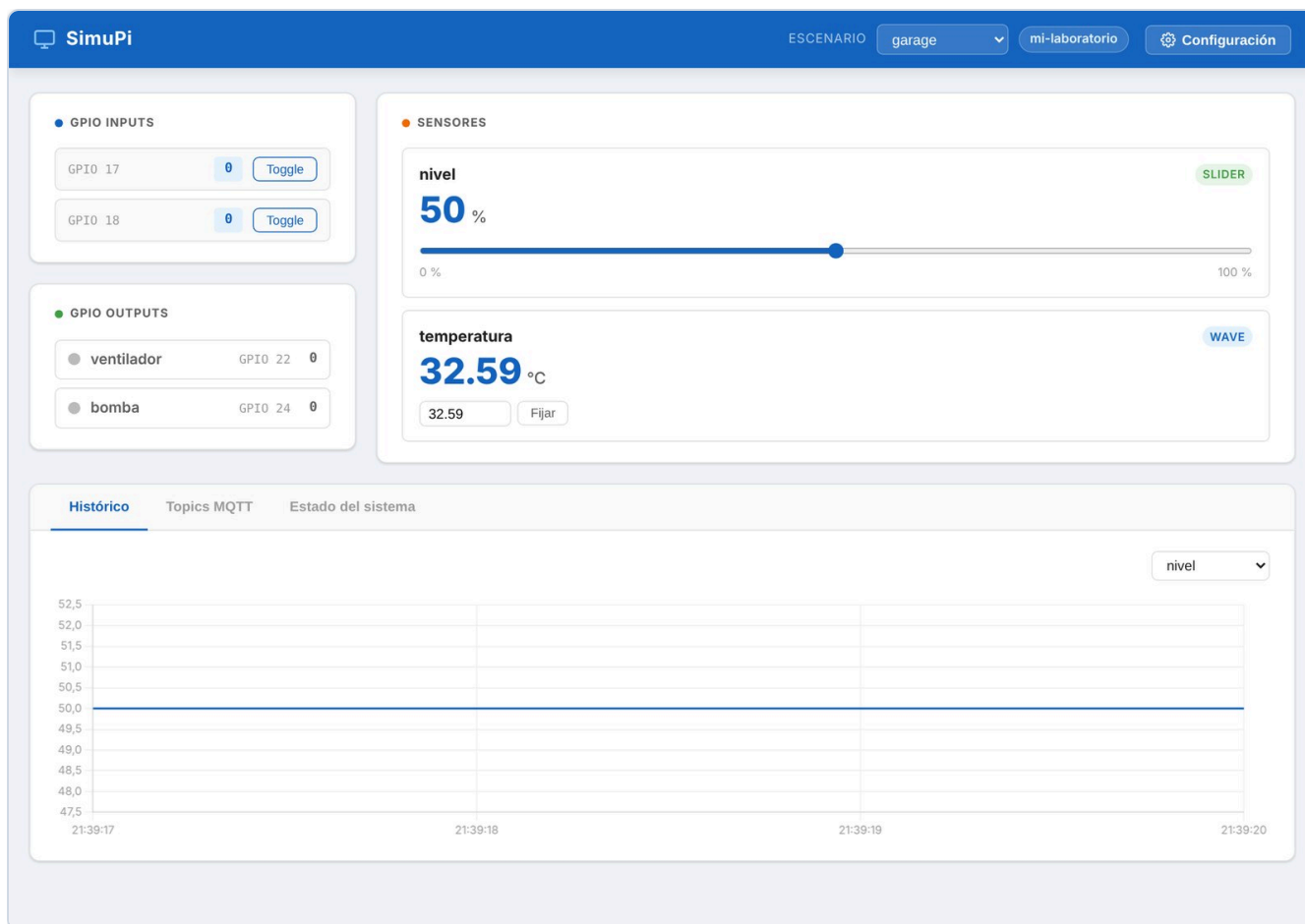


Figura 13. Resultado: el escenario *mi-laboratorio* está activo, con los alias *ventilador* y *bomba* y un sensor slider.

Mensajes de error

Si algo está mal, el mensaje aparece en rojo debajo de los botones y **no se aplica ningún cambio**.



Figura 14. Error de validación: el mínimo (40) no es menor que el máximo (15).

Mensaje	Causa y solución
JSON inválido — revisá la sintaxis	Falta una coma, una comilla o una llave. Revisá el texto.
sensor "x": min must be less than max	En ese sensor, <code>min</code> es mayor o igual que <code>max</code> .
sensor "x": invalid mode "..."	El modo está mal escrito. Usá uno de la tabla de modos.
missing 'sensors'	El JSON no tiene sensores, o está vacío.
invalid name (letters, numbers, - and _ only)	El nombre del escenario tiene espacios u otros símbolos.
Ingresá un nombre para el escenario	Apretaste Guardar con el campo de nombre vacío.

Escenarios con el mismo nombre

Si guardás un escenario con el nombre de uno de los que vienen incluidos (*garage*, *greenhouse*, *smart_home*), el tuyo tiene prioridad en la lista.

10. Qué se guarda al cerrar SimuPi

SimuPi guarda su estado automáticamente cada 5 segundos en la carpeta `data/`, al lado del ejecutable. Cuando lo volvéis a abrir, retoma donde estaba.

Archivo	Contenido
<code>data/state.json</code>	Escenario activo, valores de sensores, estado de los pines, fallas y valores fijados.
<code>data/scenarios/</code>	Los escenarios que guardaste con Guardar como escenario .

Para empezar de cero, cerrá SimuPi y borrá `data/state.json`. Para eliminar un escenario propio, borrá su archivo de `data/scenarios/`. Si trabajás en grupo, tené en cuenta que la carpeta `data/` es la de donde lo ejecutes.

11. Conectar otro programa por MQTT

Para que Node-RED (u otro cliente MQTT, como MQTT Explorer) hable con SimuPi, configurá estos datos:

Dato	Valor
Servidor / host	<code>localhost</code> (o la IP de la computadora donde corre SimuPi)
Puerto	<code>1883</code>
Usuario y contraseña	No hacen falta

Lo que sale de SimuPi se lee con un nodo **mqtt in** (`sensor/#`, `gpio/in/17` ...); las órdenes a las salidas se mandan con un nodo **mqtt out** (`gpio/out/22` ...). Los pasos detallados, con flujos de ejemplo y ejercicios, están en el *Tutorial de laboratorio SimuPi + Node-RED*.

12. Problemas frecuentes

Problema	Qué probar
Al ejecutar dice que el puerto está en uso	Hay otra copia de SimuPi abierta, o algo usa el 5000 o el 1883. Cerrá la otra copia. Para el puerto web podés usar <code>PORT=8080</code> . El 1883 no se puede cambiar: si lo usa otro broker (por ejemplo Mosquitto), cerralo.
El navegador no abre <code>localhost:5000</code>	Verificá que SimuPi siga corriendo en la terminal y que usaste el puerto correcto. Probá <code>http://localhost:5000/api/health</code> .
El gráfico del Histórico está vacío	Falta Internet (la librería se descarga al abrir la página), o recién empezó y todavía no hay lecturas. Recargá la página con conexión.
El selector muestra otro escenario que el activo	Es normal tras recargar. Fijate en el cuadro del escenario activo, al lado del selector.
Un sensor no cambia de valor	Puede estar fijado (etiqueta naranja: hacé clic en Liberar), con falla stuck u offline , o ser un slider .
Una salida no se enciende	Revisá tópico, pin y payload (ver sección GPIO Outputs). Probá enviar el mensaje desde <code>mosquitto_pub</code> para descartar un error de tu flujo.
Node-RED no recibe nada	¿Iniciaste SimuPi antes? ¿El servidor MQTT es <code>localhost:1883</code> ? ¿El tópico existe en el escenario activo (mirá Topics MQTT)?
Al reiniciar aparece otro estado distinto al esperado	SimuPi restaura el último estado guardado. Borrá <code>data/state.json</code> para empezar limpio.

A. Apéndice: referencia rápida de la API

Todo lo que hace la interfaz también se puede hacer por HTTP, útil para automatizar pruebas. Dirección base: `http://localhost:5000`.

Método	Ruta	Qué hace
GET	/api/health	Verifica que el servidor responde.
GET	/api/state	Estado completo en JSON.
GET	/api/scenarios	Lista de escenarios.
POST	/api/scenarios/load/<nombre>	Carga un escenario.
POST	/api/gpio/<pin>	Fija una entrada. Cuerpo: {"value": 1}
POST	/api/gpio/<pin>/toggle	Invierte una entrada.
POST	/api/gpio/<pin>/pulse	Pulso en una entrada. Cuerpo opcional: {"duration": 0.5}
POST	/api/sensor/<nombre>	Fija el valor de un sensor. Cuerpo: {"value": 25.0}
DELETE	/api/sensor/<nombre>	Libera un valor fijado.
POST	/api/failure/<sensor>	Cuerpo: {"mode": "stuck" "offline" "clear"}
GET	/api/history/<sensor>	Últimas 500 lecturas.
GET	/api/export/<sensor>	Descarga el historial en CSV.
GET	/api/config	Configuración del escenario activo.
POST	/api/config/apply	Aplica una configuración sin guardarla.
POST	/api/config/save/<nombre>	Guarda la configuración como escenario.